

Analiza wyników pomiarowych z prób spęczania z
wykorzystaniem aplikacji komputerowej - praca
inżynierska

Szymon Miłkowski

12.01.2026

Spis treści

1	Wstęp	2
1.1	Czego dotyczy praca	2
1.2	Opis próby spęczania	2
1.3	Opis maszyny pomiarowej	3
1.4	Aktualne rozwiązania	3
1.4.1	[nazwa programu]	3
1.5	Co zawiera praca	3
2	Cel i teza pracy	4
2.1	Cel	4
2.2	Teza	4
3	Opis narzędzi wykorzystanych do realizacji pracy	5
3.1	Język programowania	5
3.2	Środowisko graficzne	5
3.3	Kompilacja	5
4	Opis utworzonej aplikacji	6
4.1	Architektura aplikacji	6
4.1.1	Ogólna struktura aplikacji	6
4.1.2	Rysowanie wykresu	7
4.1.3	Minimalizacja renderowanych punktów	9
4.1.4	Wykrywanie zmian	10
4.2	Instrukcja obsługi	10
4.2.1	Wczytywanie plików	10
4.2.2	Obróbka wykresu	10

Rozdział 1

Wstęp

1.1 Czego dotyczy praca

Praca obejmuje stworzenie aplikacji komputerowej do graficznej i komputerowej analizy wyników pomiarowych z prób spęczania.

1.2 Opis próby spęczania

Próba spęczania polega na ściskaniu próbki wzdłuż jednej osi (pionowej) przez maszynę badawczą z zadaną prędkością. Podczas próby materiał odkształca się, zmniejszając swoją wysokość w kierunku działania siły oraz - zgodnie z zasadą zachowania objętości - zwiększając swoją szerokość w pozostałych kierunkach. Próby mogą być wykonywane w piecu oraz w temperaturze otoczenia - tzn. poza piecem.

Podczas tego procesu, materiał wewnątrz płynie, wypychając materiał na ścianach bocznych na zewnątrz, a materiał przy podstawach trze o górną i dolną ścianę maszyny, co hamuje jego przemieszczenie na zewnątrz. Efekt ten powoduje zaokrąglenie powierzchni bocznej próbki.

Wielkości mierzone przez maszynę, zawarte w pliku wyjściowym pomiaru to: czas, obciążenie, przesunięcie oraz zadany sygnał.

Po przeliczeniu wartości sił i przemieszczenia na zależność naprężenia od odkształcenia możliwe jest wyznaczenie modułu sprężystości, modułu plastyczności, granicy plastyczności, a także wytrzymałości na spęczanie.

Przeprowadzone spęczania w stanie stałocięłym pozwalają również na wyznaczenie wartości lepkości badanego materiału w funkcji prędkości ścinania. (wzory jutro)

1.3 Opis maszyny pomiarowej



Rysunek 1.1: Maszyna do przeprowadzania prób ściszania Zwick HB100

1.4 Aktualne rozwiązania

1.4.1 [nazwa programu]

Program służy do wyświetlania danych na wykresie. Posiada opcje importu danych z formatu W01, zmiany osi i serii danych oraz eksportu do pliku tekstowego CSV. Program ten jednak jest trudniejszy w obsłudze oraz przydatne funkcjonalności wymagają repetytywnych czynności ze strony użytkownika.

1.5 Co zawiera praca

Ten dokument zawiera opis problemu, cel i założenia, napotkane przeszkody, ogólną strukturę i sposób działania aplikacji oraz niektóre szczegóły implementacji.

Rozdział 2

Cel i teza pracy

2.1 Cel

Celem projektu jest stworzenie kompatybilnego i wydajnego programu, który usprawni pracę z wynikami pomiarowymi z prób spęczania. Główny nacisk kładziony jest na maksymalną oszczędność czasu i szybkość w obsłudze dla wprawionego użytkownika, co można osiągnąć przez prosty i czytelny interfejs zoptymalizowany pod najczęściej wykonywane operacje. Program wykonany w pracy ma na celu obróbkę danych pomiarowych wygenerowanych przez maszynę wytrzymałościową Zwick HB100. Surowe wyniki pomiarów zawierają wartości sił zarejestrowanych dla pozycji siłownika maszyny. W celu wyznaczenia właściwości badanego materiału dane te muszą zostać przeliczone na zależność naprężenia od odkształcenia badanej próbki. Maszyna posiada aplikację do przeglądania wyników prób oraz ich obróbki, niemniej jednak obsługa tego programu jest czasochłonna i nieintuicyjna. Zaplanowane w pracy inżynierskiej oprogramowanie ma na celu istotne skrócenie obróbki danych pomiarowych realizowane przy pomocy interaktywnych wykresów, umożliwiających jednoczesną obserwację kształtu zarejestrowanych zależności.

Dodatkowym celem jest również wysoka kompatybilność programu zarówno z nowymi jak i starszymi systemami operacyjnymi oraz wysoka wydajność nawet na starym sprzęcie komputerowym.

2.2 Teza

Stwierdzono brak powszechnego występowania prostych i darmowych aplikacji umożliwiających obróbkę danych pomiarowych z wykorzystaniem interaktywnych wykresów. W związku z powyższym sformułowano tezę, że analiza wyników pomiarowych jest możliwa dzięki takiemu podejściu, co wykazano w niniejszej pracy.

Rozdział 3

Opis narzędzi wykorzystanych do realizacji pracy

3.1 Język programowania

Wybrany językiem programowania do stworzenia aplikacji został **język C** w standardzie C11. Cechuje się on bardzo wysoką kontrolą nad działaniem programu, co pozytywnie wpływa na wydajność oraz minimalną ilość zależności, dzięki czemu łatwiej zachować kompatybilność z większą ilością systemów. Narzędzia do pracy z C pozwalają na stworzenie statycznego pliku wykonywalnego, który nie będzie potrzebować procesu instalacji.

3.2 Środowisko graficzne

Do obsługi grafiki oraz urządzeń wejścia (mysz i klawiatura) została wykorzystana **biblioteka Raylib**, posiadająca łatwe w obsłudze, bogate API kompatybilne z wieloma systemami operacyjnymi. Raylib nie posiada zależności, można ją dołączyć statycznie do pliku wykonywalnego i wymaga jedynie obsługi OpenGL w sterowniku graficznym.

3.3 Kompilacja

Prace programistyczne odbywały się na systemach Linux oraz MacOS (ARM), gdzie użyto domyślnych dla systemu kompilatorów C - odpowiednio GCC oraz Clang. Kompilacja na system Windows odbywała się na systemie Linux z wykorzystaniem MinGW. Projekt używa CMake do zarządzania kompilacją.

Rozdział 4

Opis utworzonej aplikacji

4.1 Architektura aplikacji

4.1.1 Ogólna struktura aplikacji

Komponenty

Interfejs graficzny aplikacji podzielony jest na "komponenty" - funkcje zawierające logikę kontrolną oraz rysujące pewien element interfejsu. Wywołanie tej funkcji oznacza, że komponent istnieje w obecnej klatce. Funkcje te mogą wywoływać inne funkcje komponentów, co tworzy swego rodzaju strukturę drzewa komponentów, rozpoczynającą się w głównej pętli programu. Pozycja w strukturze drzewa nie ma jednak wpływu na rozmieszczenie elementu, pozycja jest statyczna, lub określana poprzez podane parametry.

Zarządzanie stanem

Inspirowane działaniem innego drzewiastego systemu komponentów graficznych - webowym Reactem - komponenty, jeśli to konieczne, operują na dwóch rodzajach danych: właściwości (*props*) i stan (*state*).

Właściwości są podawane do funkcji komponentu jako kopia (pass by value), bądź stały wskaźnik, mają za zadanie sterować zachowaniem i wyglądem komponentu.

Stan jest podawany jako wskaźnik do pamięci zarządzanej przez rodzica (komponent wywołujący), ma za zadanie przechowywanie stanu komponentu między wywołaniami, jest modyfikowany głównie przez sam komponent oraz jego dzieci (jeśli zostanie podany do nich jako ich stan).

Stan globalny jest wykorzystywany w niektórych komponentach, które z założenia mają istnieć tylko w jednej sztuce jednocześnie, jak lista otwartych plików lub narzędzia diagnostyczne `FPSCounter` i `Clock`, jako zmienne globalne w jednostce translacyjnej (niewidoczna poza obecnym plikiem `.c`), lub statyczne zmienne lokalne w funkcji komponentu.

4.1.2 Rysowanie wykresu

Jako podstawa rysowania wykresu, została wykorzystana funkcja Rzylib `DrawSplineLinear`, łącząca podane współrzędne pikseli prostymi odcinkami. To wydajny sposób narysowania wielu prostych jednocześnie, co przy wystarczającej gęstości punktów daje iluzję krzywej.

```
DrawSplineLinear(state->point_buffer, state->visible.count, 2.f, DARKBLUE);
```

Poruszanie się w widoku

Aby zwiększyć interaktywność, użytkownik może się poruszać po płaszczyźnie XY, przeciągając płaszczyznę myszką, oraz przybliżając i oddalając kółkiem myszy. Manipulowane są w ten sposób zmienne skali oraz przesunięcia uwzględnione w przekształceniu punktów ze współrzędnych kartezjańskich na współrzędne ekranu.

Transformacja współrzędnych

Punkt podlega przekształceniu liniowemu:

$$x = \frac{z(x_p - x_v) - x_S}{x_E - x_S} * W + x_O \quad (4.1)$$

gdzie

z - współczynnik skali

x_p - współrzędna x punktu w układzie kartezjańskim

x_v - przesunięcie x w układzie kartezjańskim

x_S - współrzędna x lewej krawędzi układu kartezjańskiego przy zerowym przesunięciu

x_E - współrzędna x prawej krawędzi układu kartezjańskiego przy zerowym przesunięciu

W - szerokość wykresu w pikselach

x_O - odległość prawej krawędzi wykresu od prawej krawędzi ekranu

$$y = \frac{-z(y_p + y_v) - y_S}{y_E - y_S} * H + y_O \quad (4.2)$$

gdzie

z - współczynnik skali

y_p - współrzędna y punktu w układzie kartezjańskim

y_v - przesunięcie y w układzie kartezjańskim

y_S - współrzędna y dolnej krawędzi układu kartezjańskiego przy zerowym przesunięciu

y_E - współrzędna y górnej krawędzi układu kartezjańskiego przy zerowym przesunięciu

H - wysokość wykresu w pikselach

y_O - odległość górnej krawędzi wykresu od górnej krawędzi ekranu

Zmiana znaku y_p jest spowodowana, tym że oś $+Y$ we współrzędnych ekranowych skierowana jest w dół.

Implementacja W kodzie programu zaimplementowane jest to dwuetapowo.

Obliczanie granic widocznego fragmentu

```
Bounds compute_bounds(const float zoom, const Vector2 pan) {
    return (Bounds) {
        .start_x = PLOT_START_X / zoom + pan.x,
        .end_x = PLOT_END_X / zoom + pan.x,
        .start_y = PLOT_START_Y / zoom / ((float) PLOT_WIDTH / PLOT_HEIGHT) + pan.y,
        .end_y = PLOT_END_Y / zoom / ((float) PLOT_WIDTH / PLOT_HEIGHT) + pan.y,
    };
}
```

Transformacja pojedynczego punktu

```
Vector2 transform_v_to_pixel(const Vector2 v, const Bounds bounds) {
    return (Vector2){
        .x = (v.x - bounds.start_x)
            / (bounds.end_x - bounds.start_x)
            * PLOT_WIDTH + PLOT_OFFSET_X,

        .y = (-v.y - bounds.start_y)
            / (bounds.end_y - bounds.start_y)
            * PLOT_HEIGHT + PLOT_OFFSET_Y,
    };
}
```

Transformacja wielu punktów jednocześnie jest zoptymalizowana pod kątem ilości niepotrzebnych operacji.

4.1.3 Minimalizacja renderowanych punktów

Opis problemu

Podczas przeglądania wykresu, nie każdy punkt będzie widoczny jednocześnie. Chcąc zminimalizować ilość punktów podlegających transformacji oraz rysowaniu, należy znaleźć zakres punktów widocznych na ekranie.

Szukanie pierwszego punktu

Zakładając, że współrzędna x w zestawie danych jest niemalejąca, możemy potraktować zbiór X jako posortowany zbiór danych i użyć wydajnego algorytmu **wyszukiwania binarnego**, o złożoności obliczeniowej $O(\log_2 n)$.

```
int find_first_visible(const Vector2* data, const int count, const float start_x) {
    if (data[count - 1].x < start_x) return -1;

    int a = 0, b = count - 1;
    while (b - a > 1) {
        const int mid = (a + b) / 2;
        if (data[mid].x > start_x) b = mid;
        else a = mid;
    }
    return a;
}
```

Transformacja punktów

Po znalezieniu pierwszego elementu zbioru wyjściowego, współrzędne są przekształcane i zapisywane w tablicy wyjściowej aż do napotkania punktu znajdującego się za prawą krawędzią widoku lub końca tablicy wejściowej. Wyszukiwanie binarne nie ma tutaj zastosowania, ponieważ i tak każdy widoczny punkt musi zostać przekształcony, co wiąże się z liniowym algorytmem.

```
VisiblePointsInfo compute_visible_points_offset(
    const DataSource data_source, Vector2* point_buffer, const Bounds bounds, const Vec2 offset) {
    VisiblePointsInfo res = {
        .start = find_first_visible(data_source.data, data_source.count, bounds.start_x - offset.x),
    };
    if (res.start == -1) return res;

    const float x_factor = 1 / (bounds.end_x - bounds.start_x) * PLOT_WIDTH;
    const float y_factor = -1 / (bounds.end_y - bounds.start_y) * PLOT_HEIGHT;

    const int visible_count = data_source.count - res.start;
    for (int i = 0; i < visible_count; i++) {
        const int i_source = i + res.start;
        point_buffer[i].x = (data_source.data[i_source].x - bounds.start_x + offset.x) * x_factor;
```

```

    * x_factor + PLOT_OFFSET_X;
    point_buffer[i].y = (data_source.data[i_source].y + bounds.start_y - offset.y)
    * y_factor + PLOT_OFFSET_Y;
    if (data_source.data[i_source].x > bounds.end_x - offset.x) {
        res.count = i + 1;
        return res;
    }
}
res.count = visible_count;
return res;
}

```

Algorytm nie używa funkcji do transformacji pojedynczego punktu, co pozwala na obliczenie niezmiennych współczynników poza pętlą, unikając zbędnych operacji.

4.1.4 Wykrywanie zmian

Transformacja wszystkich widocznych punktów jest mimo wszystko kosztowną operacją, dlatego wykonywana jest jedynie gdy reprezentacja punktów na wykresie powinna ulec zmianie. Odpowiada za to zmienna *change*, zawierająca flagi sterujące rekalkulacją elementów wykresu, ustawiana przez funkcję odpowiadającą za obsługę wejścia.

4.2 Instrukcja obsługi

4.2.1 Wczytywanie plików

Wczytanie odbywa się poprzez upuszczenie jednego lub wielu plików na okno otwartego programu. Program jest w stanie wczytać dwa rodzaje plików.

Plik binarny W01

Jest to plik produkowany przez oprogramowanie maszyny Zwick HB100 i zawiera serie danych z pomiaru. Dokładny opis formatu tego pliku znajduje się w sekcji 4.3.

Podczas wczytywania tego pliku, użytkownik jest pytany o wybranie kolumn odpowiadającym osiom X i Y.

Plik tekstowy CSV

Wykres wyeksportowany do pliku CSV, może zostać otwarty ponownie w aplikacji.

4.2.2 Obróbka wykresu

Po załadowaniu wykresu, do dyspozycji użytkownika są dostępne następujące operacje:

Przesuwanie (translacja)

Trzymając klawisz Lewy Ctrl i lewy przycisk myszy, wykres przesuwany jest zgodnie z ruchem kursora myszy. Podczas przesuwania pojawi się "cień" wykresu, odpowiadający miejscu w którym będzie znajdował się wykres po przesunięciu. Puszczanie lewego przycisku myszy zatwierdza przesunięcie, a puszczenie klawisza Lewy Ctrl anuluje przesunięcie.

Blokada przesuwania

Przyciski "Blokuj X" i "Blokuj Y" blokują możliwość przesuwania wykresu wzdłuż danej osi. Jest to pomocna funkcja, jeśli użytkownik potrzebuje zmienić jedną współrzędną punktów.

Przesuwanie do zera

Przycisk "Zeruj start" przesuwa cały wykres tak, aby pierwszy punkt znajdował się w punkcie (0,0).

Odbicie

Przyciski "Odwróć X" i "Odwróć Y" odbijają daną współrzędną każdego z punktów względem zera. Odbijanie współrzędnych x dodatkowo odwraca kolejność punktów, aby zachować zasadę rosnącej współrzędnej x .

Podgląd punktu

Najeżdżając kursorem myszy na punkt wyświetla się nad nim okienko z dokładnymi wartościami współrzędnych tego punktu.